

Prompt *Engineering*

► Sabancı University ► İnanç Arın ► Module: Working with LLMs

Large language models don't run our code — they predict the next token from patterns learned in training. So how we ask is, in practice, often as decisive as what we ask. Prompt engineering is the discipline of designing those inputs so the model produces reliable, useful output. This handout walks through the foundations, then advanced techniques (in-context learning, chain prompting, chain-of-thought, self-consistency, tree-of-thought), output verification, and ends with a small RAG demo as a bridge to next week's topic.

WHAT WE'LL COVER

1. Why prompt engineering exists
2. Anatomy of a prompt: the basic ingredients
3. Instruction-based prompting (and why order matters)
4. Controlling the model: temperature, top-p, top-k
5. Advanced prompt components – Persona, Context, Format, Tone...
6. In-context learning: zero-, one-, few-shot
7. Chain prompting: decomposition
8. Chain-of-thought: think before answering
9. Self-consistency & tree-of-thought
10. Output verification: examples, grammars, JSON mode
11. Mini case study: few-shot vs. random vs. similar examples
12. Bridge to RAG (next week's preview)
13. Practical checklist

Why prompt engineering exists

Some practitioners think prompt engineering is a temporary patch — a workaround for models that haven't been aligned well enough yet. That view has merit; as models get better at instruction-following, the brittle parts of prompting will fade. But the fundamental reason prompt engineering exists is structural, not temporary:

KEY IDEA

An LLM is a conditional next-token predictor. It doesn't *execute* your request — it generates a continuation that, statistically, looks like the kind of text that follows your input. Prompt engineering is the craft of shaping that input so the most likely continuation is also the one you actually want.

Done well, prompt engineering gives us:

- **Higher accuracy** on the task we care about.
- **Lower hallucination** rates by anchoring the model to a context.
- **More consistent** outputs across repeated calls.
- **Format control** — JSON, bullets, exact field names — so downstream code can parse it.
- **Automation-readiness** for agent workflows where prompts are programmatic.

The core toolkit, which we'll unpack one by one in the rest of this handout, is: *instruction design*, *context provisioning*, *demonstrations (few-shot)*, *output structuring*, *role/persona prompting*, *constraint setting*, and *chain-of-thought style reasoning*.

§ 02

Anatomy of a prompt

At the simplest level, a prompt can be a fragment — "The sky is" — and the model will complete it ("*blue.*"). That's a pure language-modelling prompt: no instruction, just continuation. It's instructive but rarely useful for a real task.

The next step is the **instruction prompt**: an explicit task plus the data it operates on. Add an **output indicator** and you've nudged the model into producing exactly the structure you want.

INSTRUCTION	Classify the text into negative or positive.
DATA	Text: "This is a great movie!"
OUTPUT IND.	Sentiment:
→ OUTPUT	<i>Positive</i>

The **Sentiment:** indicator is doing real work — without it, the model might reply *"The text is positive"* as a full sentence. With it, the model is biased to emit a single label. This is your first taste of **format control through prompt structure**.

§ 03

Instruction-based prompting (and why order matters)

Common task categories — supervised classification, search, summarization, code generation, named entity recognition — all fit this pattern. What changes is the exact instruction and which output indicators help most. A few patterns worth internalizing:

TASK	USEFUL PROMPT ELEMENTS
Classification	Explicit label set, output indicator (Sentiment:)
Summarization	Length constraint, audience, "3 bullet points" format
NER	Entity definitions, exclusion criteria (e.g. "verbs are not entities")
Code generation	Language, function signature, expected I/O example
Translation	Source & target language tags, sometimes a glossary

Order matters

Because the model predicts left-to-right, what comes *first* conditions everything that comes after. A general rule of thumb:

Instruction first, then context. Put the data the instruction operates on immediately after the instruction. End with the output indicator.

Why? If the model sees a long block of raw data first, by the time it reaches your instruction it has already started predicting *continuation of that data*, not *execution of an instruction*. There are two well-known order effects in long prompts:

- **Primacy effect** — content near the start gets more attention.
- **Recency effect** — content near the end is also weighted heavily.
- **Lost-in-the-middle** — anything stuck in the middle of a long prompt is most likely to be ignored (Liu et al., 2023).

✗ LESS RELIABLE

Summarize the following text:
<long text...>
Explain the above in two to three sentences.

✓ MORE RELIABLE

Summarize the following text in two to three sentences:
<long text...>

The same prompt, different model: Cohere vs. OpenAI

A real, classic example: ask `Translate to Turkish.\nWhere is the nearest restaurant?`. GPT happily produces *"En yakın restoran nerede?"*. An earlier-generation Cohere model returned the puzzling *"Nearby restaurant is here."* A small reformulation — adding explicit language tags — fixed it:

```
Translate to Turkish.  
English: Where is the nearest restaurant?  
Turkish:
```

This isn't evidence that GPT is "better." It's evidence that **different models respond differently to the same prompt**, because their training data and instruction-tuning differ. Two practical implications:

1. **Future-proofing.** A prompt that works on multiple models is more durable.
2. **Cost.** If a smaller, cheaper model can do the job with a slightly more careful prompt, that's often the right trade-off.

You — the engineer — get to decide which model is right for your use case.

Controlling the model: temperature, top-p, top-k

Prompt design isn't the only knob. Every time the model picks a token, it scores all vocabulary tokens by likelihood and then *samples* from that distribution. The sampling parameters control how that draw happens.

TEMPERATURE

Scales the logits before softmax. Low temperature (`0.0` – `0.3`) → the most probable tokens dominate; output is conservative, repeatable, almost deterministic at `0` . High temperature (`0.8` – `1.2`) → less probable tokens get more chance; output is more diverse and surprising.

TOP-P (NUCLEUS SAMPLING)

Sort tokens by probability, take the smallest set whose cumulative probability $\geq p$, sample only from that set. `top_p=1.0` means consider everything; `top_p=0.1` means consider only the very top of the distribution.

TOP-K

Simpler cousin: only consider the k most probable tokens at each step. `top_k=1` is greedy decoding.

These interact. A useful mental model is: temperature reshapes the distribution, top-p / top-k truncate it. With `do_sample=False` in the HuggingFace pipeline, none of these matter — the model just picks the argmax every step.

USE CASE	TEMPERATURE	TOP-P	WHY
Brainstorming	High	High	Maximum diversity, unexpected ideas.
Email / report generation	Low	Low	Predictable, focused, conservative.
Creative writing	High	Low	Surprising word choice <i>within</i> a coherent register.

USE CASE	TEMPERATURE	TOP-P	WHY
Translation	Low	High	Faithful meaning with a wider vocabulary range.
Code generation	Low (≤ 0.2)	Low	Code rarely benefits from creativity.
Classification	0	—	You want one label, every time.

Rule of thumb: tune the prompt first, the sampling parameters second. A prompt problem is rarely fixed by raising the temperature.

§ 05

Advanced prompt components

As tasks get richer, the basic *instruction + data + output indicator* trio expands. The components below aren't a checklist you must use every time — they're a palette you mix from. The Hands-On LLM textbook (Alammar & Grootendorst, 2024) calls this out as "*the modular nature of prompting*": you add and remove components and watch their effect.

PERSONA	You are an expert in Large Language Models. You excel at breaking down complex papers into digestible summaries.
INSTRUCTION	Summarize the key findings of the paper provided.
CONTEXT	Your summary should extract the most crucial points so a researcher can quickly grasp the paper.
FORMAT	Bullet-point summary, then a concise paragraph encapsulating the main results.
AUDIENCE	Busy researchers tracking the newest LLM trends.
TOPE	Professional and clear.
DATA	<paste paper / abstract here>

Each component, briefly

PERSONA

Assigns a role. *"You are an expert in astrophysics"* primes the model toward a register and a knowledge slice. It does *not* grant the model new knowledge — but it does shift which parts of its training distribution it draws from.

INSTRUCTION

The task itself. Be specific. *"Write a description for a product"* leaves too much room. *"Write a product description in under two sentences, formal tone, no marketing superlatives"* doesn't.

CONTEXT

The *why* behind the instruction. Often this is what closes the gap between a generic answer and one tailored to your situation.

FORMAT

How the output should be structured. Without it, the model invents a format — fine for chat, deadly for an automated pipeline. JSON, Markdown table, fixed field names, numbered list — pick one and say so.

AUDIENCE

Who is this for? Calibrates technical depth, vocabulary, and tone. *"Explain it like I'm 5"* (ELI5) is genuinely effective for educational outputs.

TOPE

Formal, casual, empathetic, terse. Don't leave this implicit if it matters.

A small but underused trick: **emotional framing**. Li et al. (2023, EmotionPrompt) showed appending phrases like *"This is very important for my career"* can measurably improve quality on some benchmarks. Read that as a data point about how training data conditioning works, not a recipe — but it confirms the broader lesson that prompt content beyond the literal task carries signal.

Iterate. Always iterate.

Prompts are written like code, but tuned like experiments. The recommended workflow is: start minimal, add components only when you see the output failing in a specific way, and remove components that don't change anything. Reordering matters too — see § 03 on primacy and recency.

§ 06

In-context learning: zero-, one-, few-shot

Sometimes the cleanest way to specify a task isn't to *describe* it but to *demonstrate* it. This is called **in-context learning**: you paste a few input/output examples into the prompt, and the model picks up the pattern. The terminology comes directly from few-shot meta-learning:

TERM	WHAT IT MEANS	WHEN TO USE
Zero-shot	Instruction only, no examples	Task is generic, instruction is unambiguous
One-shot	One example of input → output	Format is unusual or very specific
Few-shot	2+ examples, often 3–8	Task is hard to specify in words, or you want the model to mimic a style

Few-shot prompting is the single most powerful trick in this handout, and it predates the chat era — the GPT-3 paper ("*Language Models are Few-Shot Learners*," Brown et al., 2020) is what put it on the map. The earlier GPT-2 paper ("*Language Models are Unsupervised Multitask Learners*," Radford et al., 2019) showed the deeper reason it works: web-scale pre-training data naturally contains things like *French: ... English: ...* pairs, so the model has effectively seen the format already.

WORKED EXAMPLE — ONE-SHOT FOR A MADE-UP WORD

USER: A 'Gigamuru' is a type of Japanese musical instrument.
An example sentence using the word Gigamuru is:
ASSISTANT: I have a Gigamuru that my uncle gave me as a gift.
I love to play it at home.
USER: To 'screeg' something is to swing a sword at it.
An example sentence using the word screeg is:

The single demonstration teaches the model what shape the answer should take. The model then produces a fluent sentence using *screeg* correctly. No instruction phrase like "use it in a sentence" was needed — the example *is* the instruction.

Zero-shot vs. few-shot — when does it actually matter?

In our own classroom experiment (see § 11 below), zero-shot GPT-4 on the AQUA-RAT-style math word problems scored **0 / 10** — not because the answers were wrong, but because they were correctly stated in a format the grader didn't recognize ("The answer is 38%" vs. ground-truth "38"). Adding three random few-shot examples brought it to **6 / 10**. This is a recurring pattern: few-shot is often less about *knowing the answer* and more about *matching the expected output shape*.

Few-shot doesn't only teach the task. It teaches the format.

Choosing examples

- **Diversity** in examples helps generalization (cover different label classes, edge cases).
- **Similarity** to the test query also helps — this is the basis of "dynamic" few-shot, where you retrieve the nearest training examples by embedding similarity. We'll see this in § 11.
- Watch for label imbalance — if 4 of 5 demonstrations have label "Yes", the model will be biased toward "Yes".
- Consistency in formatting across examples matters more than perfect content.

§ 07

Chain prompting: decomposition

When a task is too complex to do well in one shot — say, "given product features, invent a name, a slogan, and a sales pitch" — you can either ask for everything in one prompt, or break the problem into

a **pipeline** of smaller prompts whose outputs feed into one another.

```
STEP 1          features → LLM → product_name
-----
STEP 2          features + product_name → LLM → slogan
-----
STEP 3          features + product_name + slogan → LLM → sales_pitch
```

Chain prompting buys you several things:

- **Per-step parameters.** The name step can run cold (low temperature, short output); the sales pitch step can run hot and long.
- **Per-step verification.** You can check each intermediate output before passing it on.
- **Compute budget.** Each step gets the model's full attention on a smaller problem.

Common patterns this enables:

- **Response validation** — second prompt asks the model to critique the first.
- **Parallel prompts** — N prompts run simultaneously, then a merge step.
- **Long-form writing** — outline → characters → beats → dialogue.
- **RAG itself** — retrieve, then generate (we'll see this in § 12).

§ 08

Chain-of-thought: think before answering

One of the most consequential discoveries of 2022: if you ask an LLM to *show its reasoning* on multi-step problems, accuracy on those problems goes up substantially. Wei et al. (2022) called this **chain-of-thought (CoT) prompting**.

The framing matters psychologically too. Kahneman's *System 1 / System 2* distinction maps neatly onto this: a normal LLM call is fast, intuitive, single-pass — System 1. Chain-of-thought is the model's approximation of System 2: slow, deliberate, step-by-step. We're not claiming the model "really reasons" — but the output of a model that's been prompted to show its work is empirically much closer to correct on tasks that involve composition.

✗ ONE-SHOT, NO REASONING

✓ CHAIN-OF-THOUGHT

Q: Roger has 5 tennis balls. He buys 2 more cans of 3 tennis balls each.

How many does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. They used 20 to make lunch and bought 6 more. How many apples do they have?

A: The answer is 27. $\times$

Q: Roger has 5 tennis balls. He buys 2 more cans of 3 tennis balls each.

How many does he have now?

A: Roger started with 5 balls.

2 cans of 3 balls is 6 balls.

$5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. They used 20 to make lunch and bought 6 more. How many apples do they have?

A: $23 - 20 = 3$. $3 + 6 = 9$.

The answer is 9. $\checkmark$

The mechanism is intuitive once you accept the next-token-prediction frame: each intermediate reasoning token gives the model a foothold for the next one. Computing " $23 - 20 = 3$ " as text reduces the next prediction to a much easier problem. Without it, the model has to compress the entire calculation into the very few tokens of "The answer is ____", which is harder.

Zero-shot CoT

You don't always have to provide reasoning examples. Kojima et al. (2022) showed that simply **appending the phrase "*Let's think step by step.*"** unlocks similar reasoning behavior — no demonstrations required.

ZERO-SHOT CoT PROMPT

The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?
Let's think step by step.

Output: "*Step 1: 23 apples. Step 2: subtract 20 $\rightarrow$ 3. Step 3: add 6 $\rightarrow$ 9. The answer is 9.*"

Variants that also work, sometimes better depending on the model:

- "*Take a deep breath and work through this step by step.*" (Yang et al., 2023)
- "*Let's work through this problem step by step.*"
- "*Show your work.*"

Modern reasoning models (OpenAI's o1/o3 series, DeepSeek-R1, etc.) bake chain-of-thought into the model's training itself, so you may notice that asking them to "think

step by step" makes less of a difference than with a vanilla instruction-tuned model.
The technique didn't go away — it moved inside.

§ 09

Self-consistency & tree-of-thought

Self-consistency: sample many, vote

Once you allow non-zero temperature, the same CoT prompt can produce different reasoning chains and sometimes different final answers. Wang et al. (2022) proposed a simple fix: **run the prompt N times** and **take the majority answer**. This is called **self-consistency**.

SAMPLE 1	2 cans × 3 balls = 6 balls. 5 + 6 = 11. → 11
SAMPLE 2	If he buys 2 cans of 3, he has 2 × 3 = 6. → 6 (wrong)
SAMPLE 3	2 cans of 3 = 6. Add 5. → 11
VOTE	Majority answer: 11 ✓

Trade-off: it's n times slower and more expensive. But on hard reasoning benchmarks, the accuracy lift is real. A common practical setting is $N = 5$ with temperature 0.7.

Tree-of-thought: from a chain to a search

Self-consistency runs the same prompt N times *independently*. The chains never talk to each other; we just trust majority voting at the end. But what if a reasoning chain takes a wrong turn early on? It cannot recover — autoregressive generation only ever moves forward, one token at a time.

Yao et al. (2023) reframed reasoning as something the classical AI tradition has studied for sixty years: **search in a state space**. A "thought" is not just one token — it is a coherent intermediate step that can be evaluated, branched from, or abandoned. Stack thoughts into a tree, and the model can *look ahead*, *compare alternatives*, and *backtrack* when a path looks dead. CoT is a chain; ToT is the tree that contains many such chains plus the machinery to choose between them.

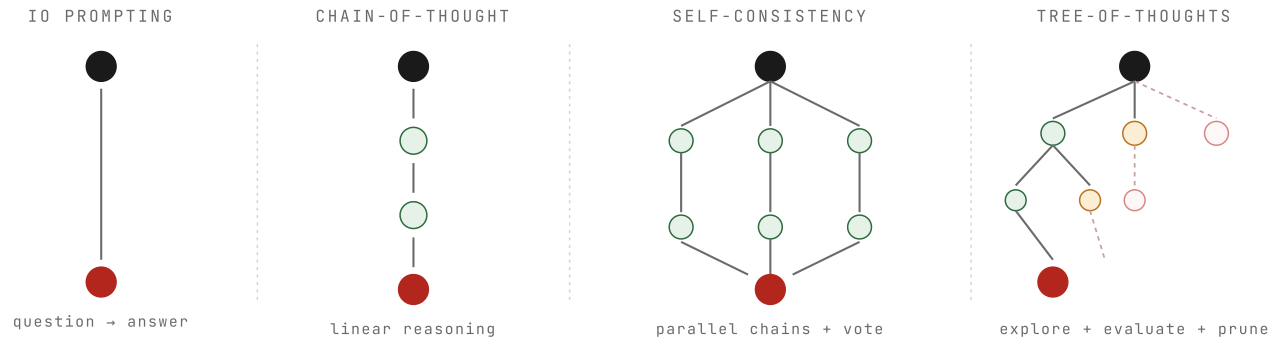


FIGURE 9.1 — FROM SINGLE CHAIN TO SEARCH TREE. **GREEN**: PROMISING THOUGHT. **AMBER**: UNCERTAIN. **FADED RED**: PRUNED DEAD-END. ONLY TOT PRUNES.

THE FOUR DESIGN QUESTIONS

Yao et al. distill ToT into four design choices. Every concrete instantiation — from Game of 24 to creative writing — answers each of them differently.

1 • DECOMPOSE

What counts as one "thought"? *Granularity* matters: a single arithmetic step (Game of 24), a paragraph plan (creative writing), a guessed word (crosswords). Too small and the tree explodes; too large and the evaluator cannot judge it.

2 • GENERATE

From a state, how do we get k candidate next thoughts? Two options: **sample** k i.i.d. completions of the same CoT prompt (good for rich, open-ended spaces) or **propose** — a single prompt that asks the model to enumerate k distinct next steps (good for constrained spaces).

3 • EVALUATE

Is this state worth pursuing? **Value**: rate each state independently (e.g. label "sure / likely / impossible"). **Vote**: show several states side by side and ask the model to pick the best — useful when absolute scoring is hard, e.g. judging which essay plan is more coherent.

4 • SEARCH

How do we walk the tree? **BFS** with beam width b (Game of 24, crosswords) or **DFS** with backtracking when a branch falls below threshold (creative writing). Classical AI search algorithms; the model is the heuristic.

THE HIDDEN ASSUMPTION

Step 3 is the crux of ToT — and its biggest fragility. The same model that *generates* the thoughts also *evaluates* them. If self-evaluation is unreliable (as it tends to be on smaller models, or on tasks far from the model's training distribution), the search heads in the wrong direction with high confidence. ToT works because GPT-4-class models are decent self-judges; do not assume the same of a 7B model.

A worked example: Game of 24

The cleanest illustration in the paper. You're given four numbers — say 4, 9, 10, 13 — and must combine them with +, −, ×, ÷ to make 24. Three intermediate states (after applying three binary operations) lead to a single number that you check against 24.

The numbers behind the technique:

METHOD	GAME OF 24 SUCCESS	RELATIVE COST
GPT-4 + IO prompting	7.3 %	1×
GPT-4 + CoT	4.0 %	~1×
GPT-4 + CoT-SC (Self-Consistency, 100 samples)	9.0 %	~100×
GPT-4 + ToT (b = 5, BFS)	74 %	~5.5× (vs CoT)

An almost twenty-fold lift from a search wrapper around the same model. Notice that pure self-consistency barely helps here — voting over independent linear chains cannot rescue you from a hard combinatorial problem. You need branching with pruning, which is exactly what ToT provides.

START

4, 9, 10, 13
target = 24

STEP 1

13 - 9 = 4
left: 4, 4, 10 ✓ sure

10 - 4 = 6
left: 6, 9, 13 ✓ sure

13 - 10 = 3
left: 3, 4, 9 ~ likely

9 + 10 = 19
left: 4, 13, 19 x

4 × 13 = 52
x pruned

STEP 2

4 + 4 = 8
8, 10 ~

10 - 4 = 6
4, 6 ✓ sure

9 + 13 = 22
6, 22 ~

6 × 9 = 54
x

3 × 4 = 12
9, 12 ~

4 + 9 = 13
x

STEP 3

8 + 10 = 18
x ≠ 24

4 × 6 = 24
✓ goal!

22 - 6 = 16
x

12 - 9 = 3
x

(13 - 9) × (10 - 4) = 24

FIGURE 9.2 – GAME OF 24 WITH TOT (BFS, BEAM = 3). AT EACH TIER THE MODEL **PROPOSES** SEVERAL ARITHMETIC STEPS, THE SAME MODEL **EVALUATES** THE RESULTING STATE ("SURE / LIKELY / IMPOSSIBLE"), AND ONLY THE TOP CANDIDATES SURVIVE INTO THE NEXT TIER. FADED RED = PRUNED. THE WINNING CHAIN AT THE BOTTOM IS JUST ONE PATH THROUGH THIS TREE – THE REST IS THE *COST* OF FINDING IT.

WHAT THE SEARCH LOOKS LIKE, PROMPT BY PROMPT

The two prompts that drive the whole search — one to generate, one to evaluate:

PROPOSE-PROMPT (GENERATOR)

```
Input: 4 9 10 13
Possible next steps:
4 + 9 = 13   (left: 10 13 13)
10 - 4 = 6   (left: 6 9 13)
13 - 9 = 4   (left: 4 4 10)
13 - 10 = 3  (left: 3 4 9)
9 + 10 = 19  (left: 4 13 19)
...
```

VALUE-PROMPT (EVALUATOR)

Evaluate if given numbers can reach 24
(sure / likely / impossible).

```
10 14:    10 + 14 = 24.  → sure
4 4 10:    4 + 4 + 10 = 18; 4 × 4 - 10 = 6;
           (10 - 4) × 4 = 24.  → sure
4 9 11:    9 + 11 + 4 = 24.  → sure
1 3 3:    1 × 3 × 3 = 9; (1 + 3) × 3 = 12.  → impossible
...
Now evaluate: {numbers}
```

Both prompts run on the *same model*, with different system messages. The "intelligence" of the search is entirely in how well the model rates partial states it has never been explicitly trained to rate.

Costs and when not to use ToT

✗ WHEN ToT IS OVERKILL

- The task is one-shot factual: *"Translate this sentence."*
- Linear CoT already solves it reliably.
- You're using a small or weak model — its self-evaluation will mislead the search.
- Latency matters more than accuracy (chatbots, autocomplete).

✓ WHEN ToT PAYS OFF

- The problem genuinely branches: planning, combinatorial puzzles, constrained generation.
- A single mistake early ruins the whole answer.
- You can afford 5–100× the inference cost of CoT.
- The model is strong enough to evaluate its own intermediate states.

ToT is the academic ancestor of the "reasoning models" you now use through APIs. o1/o3, DeepSeek-R1, Gemini-Thinking — they all spend extra inference compute on some form of search-with-evaluation. The difference is that ToT externalizes the search in a Python loop with prompts, whereas reasoning models have learned to do it inside a single forward pass.

The lightweight version: a "panel of experts" prompt

For everyday use, you rarely want to implement the full search. A surprisingly effective approximation captures most of the benefit in a single API call:

ZERO-SHOT TREE-OF-THOUGHT (THE LIGHTWEIGHT VERSION)

Imagine three different experts are answering this question. All experts will write down 1 step of their thinking, then share it with the group. Then all experts will go on to the next step, etc. If any expert realizes they're wrong at any point, they leave.

The question is: <your question>

Same model, single call, but the simulated multi-perspective discussion often catches errors that a single linear chain would miss. It is not a real tree search — there's no external evaluator, no backtracking — but the prompt *scaffolds* the model into producing branching reasoning that converges by elimination.

BIGGER PICTURE

Step back and look at the trajectory: **IO** → **CoT** → **Self-Consistency** → **ToT** → **Graph-of-Thoughts** (Besta et al., 2024, where thoughts can also be *merged*, not just extended). Each step trades inference compute for accuracy. This is exactly the axis along which "reasoning models" are now scaling — and the open research question is how much of this should be a Python wrapper around a model versus learned behaviour inside the model itself. A particularly open sub-question for your own work: how does self-evaluation degrade when the model is small, or when the task is in a low-resource language like Turkish?

§ 10

Output verification

Once your prompt is going into a *system*, "looks right to a human" isn't good enough. Four properties typically matter:

PROPERTY	WHAT CAN GO WRONG
Structured	Output should be valid JSON / XML / a known schema; instead the model wraps it in prose or invents fields.
Valid	Among allowed labels {positive, neutral, negative} the model emits "mostly positive."
Ethical	PII leakage, profanity, bias, stereotyping.
Accurate	Hallucinated facts that look authoritative.

Three families of techniques to address these:

1 • EXAMPLES (FEW-SHOT FOR FORMAT)

Show the exact JSON or label format you want. The model imitates the structure. Cheap, simple, but the model can still drift.

2 • CONSTRAINED SAMPLING / GRAMMARS

At sampling time, mask out any token that would violate your schema. If the JSON spec says the next token must be `"`, only tokens starting with `"` are allowed. Libraries: `llama-cpp-python`'s `response_format={"type": "json_object"}`, Outlines, Guidance, Guardrails, LMQL, and OpenAI's *Structured Outputs*.

3 • FINE-TUNING

Train the model itself on data that matches your expected output. Most powerful, most expensive. Covered later in the course.

Few-shot for format — concrete example

✗ ZERO-SHOT — DRIFTS AND TRUNCATES

✓ ONE-SHOT — CLEAN STRUCTURE

Create a character profile for an RPG game in JSON format.

```
→ {
  "characterProfile": { ... },
  "attributes": {
    "strength": 10,
    "dexterity": 17,
    "constitution": 12,
    "intelligence": 12,
    "wisdom": 10,
    "charisma": ← truncated
  }
}
```

Create a short character profile.

Use only this format:

```
{
  "description": "...",
  "name": "...",
  "armor": "...",
  "weapon": "..."
}
→ {
  "description": "A cunning rogue...",
  "name": "Lysandra Shadowstep",
  "armor": "Leather Cloak of the Night",
  "weapon": "Dagger of Whispers"
}
```

JSON mode — guaranteed parseability

When you literally cannot afford an unparseable response, use the model's native JSON mode. With

`llama-cpp-python`:

```
output = llm.create_chat_completion(
    messages=[
        {"role": "user",
         "content": "Create a warrior for an RPG in JSON format."}
    ],
    response_format={"type": "json_object"},
    temperature=0,
)['choices'][0]['message']['content']

import json
parsed = json.loads(output)  # guaranteed to succeed
```

The grammar machinery rejects, at sampling time, any token that would break JSON syntax. The model's choice is constrained, but the *content* within the schema is still its own. OpenAI's API has the equivalent `response_format={"type": "json_schema", "json_schema": ...}` with a full schema, which is even stricter.

§ 11

Mini case study — math word problems

Putting several techniques together on a real benchmark. Dataset: [AQuA-RAT-style](#) math word problems (multiple-choice arithmetic, train $\approx$ 30k, test $\approx$ 3k). We grade **exact match on the final string** — so a correct numerical answer formatted differently still counts as wrong.

SETUP	EXAMPLES	SCORE (N=10)	WHAT BROKE / HELPED
1 • Zero-shot	0	0 / 10	Answers were often <i>correct</i> but wrapped in prose: " <i>The percentage of profit would have been 38%</i> ". Format mismatch.
2 • Few-shot, random	3 random training examples	6 / 10	Examples taught the model the expected output shape (" <i>Answer: 38</i> "). Most failures were on harder problems.
3 • Few-shot, similar	3 nearest-neighbour examples by embedding similarity	5 / 10	Sometimes <i>worse</i> than random — the retrieved examples used decorations like " <i>option C</i> " that bled into the model's response.

Three lessons:

1. **"Wrong" can mean format, not knowledge.** Always look at the failure cases. The model knew the answers in setup 1 — it just didn't know how to deliver them.
2. **Few-shot is mostly about output shape**, especially for instruction-tuned models that are already good at the task.
3. **Similar examples are not automatically better.** If they bring along stylistic noise (like option-letter labels), the model imitates that too. Diversity often beats similarity.

The retrieval pattern in setup 3 — embed the test query, search a vector index of training examples, paste the top-k into the prompt — is, structurally, RAG.

§ 12

Bridge to RAG (next week)

Retrieval-Augmented Generation is two steps glued together: *retrieve* the most relevant context from a knowledge base, then *generate* the answer conditioned on that context. The "generate" step is pure prompt engineering — every technique in this handout still applies.

1 • INDEX Embed every doc with `text-embedding-3-small` ; store vectors in FAISS.

2 • RETRIEVE Embed user query, take top-k documents by cosine similarity.

Build a prompt with: query + retrieved context + similarity score + instruction. Send to LLM.

The generation prompt below is a small but real RAG pattern from a Turkish customer support FAQ system. Notice that it explicitly instructs the model on what to do when the retrieved similarity is too low — preventing it from confabulating an answer outside its grounded knowledge.

RAG GENERATION PROMPT — THREE BRANCHES

Aşağıdaki bilgilere göre kullanıcıyı bilgilendir.

[BRANCH 1 — high similarity]

Input: <user query>

Context: <top-1 retrieved doc>

Skor: <cosine similarity, e.g. 0.78>

url: <source url>

Asistan düşüncesi: Cevap verecek yeterince bilğim var.

Asistan cevabı: <grounded answer + URL citation>

[BRANCH 2 — low similarity]

Asistan düşüncesi: Cevap verecek yeterince bilğim yok.

Asistan cevabı: "Bu konuda elimde yeterli bilgi yok..."

[BRANCH 3 — no match at all]

Asistan düşüncesi: Veritabanımda benzer hiçbir bilgi yok.

Asistan cevabı: Politely refuse + suggest alternative resources.

Try it: ask the bot something it has indexed ("How is daily average consumption calculated?") and it answers with the source URL. Ask something it hasn't ("What was last night's Fenerbahçe–Rizespor score?") and the low similarity score routes it to branch 3 — the model honestly says "I don't know." Without that branching logic, the model would happily invent a score.

RAG is not magic. It's prompt engineering with a smarter way of choosing what to put in the context window.

Practical checklist

Before shipping a prompt to production — or even submitting it as homework — walk through this list:

SPECIFICITY

Have I told the model exactly what I want — length, format, vocabulary, tone? Could a careful colleague execute this prompt without asking me follow-up questions?

ORDER

Is the instruction at the top? Is the data right after it? Is the output indicator the very last thing the model sees before generating?

EXAMPLES

If the format is unusual, do I have at least one example? Are my examples diverse and representative? No accidental label imbalance?

SAMPLING

Have I matched temperature/top-p to the task? (Classification → 0. Brainstorm → high.) Am I using JSON mode if I will parse the output programmatically?

VERIFICATION

How will I detect failure? Do I have a fallback for hallucination? For long pipelines, can I validate intermediate steps?

REASONING

Is this a multi-step problem? Should I use chain-of-thought, self-consistency, or decompose into a chain of prompts?

KEY IDEA

Prompt engineering is not a list of tricks. It's the practical art of communicating with a system that has read most of the internet but knows nothing about your specific task. Every technique in this handout is a way of giving that system enough context to do the right thing.

Further reading

- Brown et al. (2020). *Language Models are Few-Shot Learners*. NeurIPS.
- Wei et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS.
- Kojima et al. (2022). *Large Language Models are Zero-Shot Reasoners*. NeurIPS.
- Wang et al. (2022). *Self-Consistency Improves Chain of Thought Reasoning*. ICLR 2023.
- Yao et al. (2023). *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS.
- Besta et al. (2024). *Graph of Thoughts: Solving Elaborate Problems with Large Language Models*. AAAI.
- Liu et al. (2023). *Lost in the Middle: How Language Models Use Long Contexts*. arXiv:2307.03172.
- Li et al. (2023). *EmotionPrompt: Leveraging Psychology for LLM Enhancement*. arXiv:2307.11760.
- Yang et al. (2023). *Large Language Models as Optimizers*. arXiv:2309.03409.
- Alammari & Grootendorst (2024). *Hands-On Large Language Models*. O'Reilly. — Chapter 6 in particular.
- Anthropic. *Prompt Engineering Overview*. docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview
- OpenAI. *GPT-4.1 Prompting Guide*. Available on OpenAI cookbook.